

Nearest Neighbour Algorithms

Christos Dimitrakakis

September 30, 2026

Outline

Introduction

The hidden secret of machine learning

The algorithm

k Nearest Neighbours

Extensions and parameters

Activities

Activity 1: kNN parameters

ACTIVITY

Activity 2: Predictions vs Decisions

ACTIVITY

Homework

Introduction

The hidden secret of machine learning

The algorithm

k Nearest Neighbours

Extensions and parameters

Activities

Activity 1: kNN parameters

ACTIVITY

Activity 2: Predictions vs Decisions

ACTIVITY

Homework

Supervised learning

- ▶ Given labelled training examples $(x_1, y_1), \dots, (x_T, y_T)$ where
- ▶ $x_t \in X$ are **features**
- ▶ $y_t \in Y$ are **labels**..

Feature space $\mathcal{X}$

- ▶ Usually $\mathcal{X} = \mathbb{R}^n$: the n-dimensional Euclidean space
- ▶ How do we use your class data?

Classification

- ▶ $Y = \{1, \dots, m\}$ are **discrete** labels

Regression

- ▶ $Y = \mathbb{R}^m$ are **continuous** values

The kNN algorithm idea

- ▶ Assume an unknown example is similar to its neighbours
- ▶ Smoothness allows us to make predictions

Discriminatory analysis-nonparametric discrimination: consistency properties, Evelyn Fix and Joseph L. Hodges Jr, 1951.

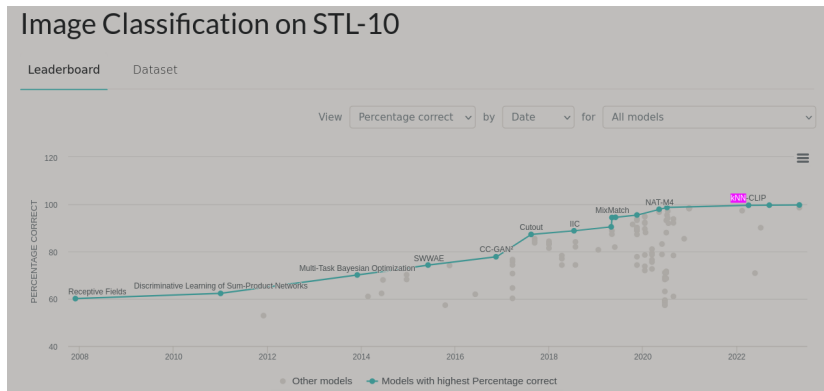


Figure: Evelyn Fix



Figure: Joseph Hodges

Performance of KNN on image classification



- ▶ Really simple!
- ▶ Can outperform really complex models!

Introduction

The hidden secret of machine learning

The algorithm

k Nearest Neighbours

Extensions and parameters

Activities

Activity 1: kNN parameters

ACTIVITY

Activity 2: Predictions vs Decisions

ACTIVITY

Homework

The Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d
- ▶ $t^* = \arg \min_t d(x_t, x)$ // How do we implement this?
- ▶ Return $\hat{y}_t = y_{t^*}$

Classification

$$\hat{y}_t \in [m] \equiv \{1, \dots, m\}$$

Nearest Neighbour Algorithms

- └ The algorithm
 - └ k Nearest Neighbours
 - └ The Nearest Neighbour algorithm

The Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_i, y_i)_{i=1}^n$, test point x , distance d
- ▶ $i^* = \arg \min_i d(x_i, x)$ // How do we implement this?
- ▶ Return $\hat{y}_i = y_{i^*}$

Classification

$\hat{y}_i \in [m] = \{1, \dots, m\}$

Explanation of $\arg \max$. If $S = \{x_1, \dots, x_n\}$ is a set of elements with the $>$ and $=$ relations, then:

- $\max S \geq x_i \quad \forall x_i \in S$.
- $\exists x_i \in S$ s.t. $\max S = x_i$.

In addition

- $\arg \max S$ are the **indices** i for which $\max S = x_i$.
- The converse relation applies to $\arg \min$.

Labels: In a classification problem, we need to predict one of m possible labels. It is customary to use the numbers $\{1, \dots, m\}$ as the set of labels, also denoted $[m]$ for brevity.

In a regression problem, we need to predict an m -dimensional real-valued variable, i.e. a value in $\mathbb{R}^m$. In simple regression, $m = 1$.

The k -Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d , neighbours k
- ▶ Calculate $h_t = d(x_t, x)$ for all t .
- ▶ Get sorted indices $s = \text{argsort}(h)$ so that $d(x_{s_i}, x) \leq d(x_{s_{i+1}}, x)$ for all i .
- ▶ Return $\sum_{i=1}^k y_{s_i} / k$.

Classification

- ▶ We use a **one-hot encoding** $(0, \dots, 0, 1, 0, \dots, 0)$, with $y_t \in \{0, 1\}^m$.
- ▶ The class of the t -th example is $j \Leftrightarrow y_{t,j} = 1$.
- ▶ Equivalently, return p with

$$p_i = \sum_{t=1}^k \mathbb{I}\{y_{s_t} = i\} / k$$

Regression

- ▶ $y_t \in \mathbb{R}^m$, so we need do nothing

Nearest Neighbour Algorithms

The algorithm

k Nearest Neighbours

The k -Nearest Neighbour algorithm

The k -Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_i, y_i)_{i=1}^n$, test point x , distance d , neighbours k
- ▶ Calculate $h_i = d(x_i, x)$ for all i .
- ▶ Get sorted indices $s = \text{argsort}(h)$ so that $d(x_{s_i}, x) \leq d(x_{s_{i+1}}, x)$ for all i .
- ▶ Return $\sum_{i=1}^k y_{s_i} / k$.

Classification

- ▶ We use a **one-hot encoding** $(0, \dots, 0, 1, 0, \dots, 0)$, with $y_i \in \{0, 1\}^m$.
- ▶ The class of the i -th example is $j \Leftrightarrow y_{i,j} = 1$.
- ▶ Equivalently, return p with

$$p_i = \sum_{t=1}^k \mathbb{1}\{y_{s_t} = i\} / k$$

Regression

- ▶ $y_i \in \mathbb{R}^m$, so we need do nothing

Multiple neighbours. We need multiple neighbours to smooth out the influence of spurious closest neighbours. To do so, we sort all the neighbours according to their distance, so that we was.

In particular, if we are sorting an array z then $s = \text{argsort}(z)$ returns the indices s so that $z[s[i]] > z[s[i+1]]$

The number of neighbours

Discuss in groups of 2-3 people. Write down your answers.

$$k = 1$$

- ▶ How does it perform on the training data?
- ▶ How might it perform on unseen data?

$$k = T$$

- ▶ How does it perform on the training data?
- ▶ How might it perform on unseen data?

Distance function

For data in $\mathbb{R}^n$, p -norm

$$d(x, y) = \|x - y\|_p$$

Scaled norms

When features having varying scales:

$$d(x, y) = \|Sx - Sy\|_p$$

Or pre-scale the data

Complex data

- ▶ Manifold distances
- ▶ Graph distance

Nearest Neighbour Algorithms

- └ The algorithm
 - └ Extensions and parameters
 - └ Distance function

Distance function

For data in $\mathbb{R}^n$, p -norm

$$d(x, y) = \|x - y\|_p$$

Scaled norms

When features having varying scales:

$$d(x, y) = \|Sx - Sy\|_p$$

Or: pre-scale the data

Complex data

- Manifold distances
- Graph distance

Data pre-scaling

$$\|Sx - Sy\|_2 = \left\| \sum_i Sx - Sy \right\|$$

Distances

A distance $d(\cdot, \cdot)$:

- ▶ Identity $d(x, x) = 0$.
- ▶ Positivity $d(x, y) > 0$ if $x \neq y$.
- ▶ Symmetry $d(y, x) = d(x, y)$.
- ▶ Triangle inequality $d(x, y) \leq d(x, z) + d(z, y)$.

For data in $\mathbb{R}^n$, ℓ_p -norm

$$d(x, y) = \|x - y\|_p$$

Norms

A norm $\| \cdot \|$

- ▶ Zero element $\|0\| = 0$.
- ▶ Homogeneity $\|cx\| = c\|x\|$ for any scalar a .
- ▶ Triangle inequality $\|x + y\| \leq \|x\| + \|y\|$.

p -norm

$$\|z\|_p = \left(\sum_i z_i^p \right)^{1/p}$$

Neighbourhood calculation

If we have T datapoints

Sort and top K .

- Requires $O(T \ln T)$ time

Use the Cover-Tree or KD-Tree algorithm

- Requires $O(cK \ln T)$ time.
- c depends on the data distribution.

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = \mathbb{P}(y = i|x),$$

where $\mathbb{P}(y = i|x)$ is the probability that y is i , given x .

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = \mathbb{P}(y = i|x),$$

where $\mathbb{P}(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = \mathbb{P}(y = i|x),$$

where $\mathbb{P}(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i \mathbb{P}(y_t = i|x_t)$

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = \mathbb{P}(y = i|x),$$

where $\mathbb{P}(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i \mathbb{P}(y_t = i|x_t)$
- ▶ We observe the actual label y_t .

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = \mathbb{P}(y = i|x),$$

where $\mathbb{P}(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i \mathbb{P}(y_t = i|x_t)$
- ▶ We observe the actual label y_t .
- ▶ We **win** if $y_t = a_t$ and **lose** otherwise

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = \mathbb{P}(y = i|x),$$

where $\mathbb{P}(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i \mathbb{P}(y_t = i|x_t)$
- ▶ We observe the actual label y_t .
- ▶ We **win** if $y_t = a_t$ and **lose** otherwise

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = \mathbb{P}(y = i|x),$$

where $\mathbb{P}(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i \mathbb{P}(y_t = i|x_t)$
- ▶ We observe the actual label y_t .
- ▶ We **win** if $y_t = a_t$ and **lose** otherwise

The model versus the prediction

- ▶ The **model** P tells us the probability of different classes.
- ▶ When we **decide** what our prediction should be, we can **use** the model.
- ▶ We will use π to denote the **decision rule** or **policy**.

Decisions versus predictions

- ▶ We frequently need to make a **decision**, instead of just a **prediction**.
- ▶ Our **utility** function $U(y, a)$ represents our **preferences**.
- ▶ The space of **actions** A is *not identical* to the set of **labels** Y .

Decisions versus predictions

- ▶ We frequently need to make a **decision**, instead of just a **prediction**.
- ▶ Our **utility** function $U(y, a)$ represents our **preferences**.
- ▶ The space of **actions** A is *not identical* to the set of **labels** Y .

Minimise spam annoyance

What utility function would you use for the spam detection problem?

Utility	Pass	Flag	Trash
Normal			
Spam			
Virus			

Decisions versus predictions

- ▶ We frequently need to make a **decision**, instead of just a **prediction**.
- ▶ Our **utility** function $U(y, a)$ represents our **preferences**.
- ▶ The space of **actions** A is *not identical* to the set of **labels** Y .

Minimise spam annoyance

What utility function would you use for the spam detection problem?

Utility	Pass	Flag	Trash
Normal			
Spam			
Virus			

Classification decision to maximise expected utility

- ▶ Expected utility of a single decision

$$\mathbb{E}[U|a, x] = \sum_y \mathbb{P}(y|x, a) U(y, a) = \sum_y \mathbb{P}(y|x) U(y, a)$$

- ▶ The decision maximising expected utility: $a^* = \arg \max_a \mathbb{E}[U|a, x]$

Introduction

The hidden secret of machine learning

The algorithm

k Nearest Neighbours

Extensions and parameters

Activities

Activity 1: kNN parameters

ACTIVITY

Activity 2: Predictions vs Decisions

ACTIVITY

Homework

Using kNN in scikitlearn

Setup

- ▶ Use the class dataset from earlier (or another dataset given in class).
- ▶ Use scikit-learn's `KNeighborsClassifier` (or `KNeighborsRegressor` if your target is continuous).

The effect of k and d

- ▶ Fit the model for several values of k (e.g. $k = 1, 3, 5, 15, T$), recording both **training** and **test** accuracy for each.
- ▶ Rescale the features (e.g. with `StandardScaler`) and repeat. Does the ranking of neighbours change? Does performance change?
- ▶ Instead of rescaling the features, change the **distance** function. Can you change it so it has the same effect?

Behaviour of k :

- ▶ Argue why the **training** accuracy of k -NN is always exactly 100% when $k = 1$.
- ▶ What is the training accuracy when $k = T$, assuming n_i examples in class i ?
- ▶ Does the same hold for **test** accuracy when $k = 1$? How about $k = T$? Check this against the plot you made.

Setup

In a group, choose a utility matrix $U(y, a)$ for the spam example from the slides, by filling in the table:

Utility	Pass	Flag	Trash
Normal			
Spam			
Virus			

The effect of different utilities

- ▶ Fit a kNN classifier and call `predict_proba()` to get $\mathbb{P}(y \mid x)$ for each test point.
- ▶ Write a function `utilityScore(y, actions, U)` that takes the true labels y_t and chosen actions a_t , and returns $\sum_{t=1}^T U(y_t, a_t) / T$.
- ▶ Write a second function `utilityAction(probs, U)` that returns, for each test point, the action a^* maximising expected utility under $\mathbb{P}(y \mid x)$ and your U .
- ▶ Compare the average utility of plain `predict()` (most likely class) against your `utilityAction()`.

The math behind predictions and decisions

- ▶ Once you have $a^* = \arg \max_a \sum_y \mathbb{P}(y|x) U(y, a)$, **prove** that no other decision rule can achieve higher expected utility under your model.
- ▶ Show that if $A = Y$ and if $U(y, a) = \mathbb{I}\{y = a\}$ (you only care about being right), then a^* is the most likely class, $\arg \max_y \mathbb{P}(y | x)$.
- ▶ Bonus: What changes once $\mathbb{P}(y|x)$ is **estimated** (e.g. via kNN) rather than known exactly? For example, say that $P^*(y|x)$ is the true model.

Exercise 1

- ▶ Define a method to create an arbitrary amount of **synthetic** data for two classes. Call this method `[x, y] = generate_data(n_dim, n_data)` so that it generates `n_data` points, with `n_dim` dimensions per point. One way to do that is to generate points from two gaussians.
- ▶ Calculate the accuracy for 100 different synthetically generated training and testing datasets as k increases.
- ▶ Average the training and testing accuracy over these 100 runs.
- ▶ Plot the resulting curves. What do you see?